

1. Marco teórico

1.1. Single Value Decomposition (SVD)

En el marco de la *principal component analysis* (PCA), para extraer los "componentes principales" de la entrada, se utilizan diversos métodos matemáticos. La *single value decomposition* (SVD) es uno de los métodos más utilizados dada su ligereza computacional en comparación a los métodos teóricos tradicionales propuestos en la definición de PCA.

Esencialmente, la SVD busca descomponer una matriz de entrada $X \in \mathbb{R}^{m \times n}$ en una multiplicación de tres submatrices:

$$X = U\Sigma V^T \quad (1)$$

en la cual U corresponde a los *vectores singulares izquierdos*, Σ corresponde a los *valores singulares* y V corresponde a los *vectores singulares derechos*.

- U es una matriz ortogonal de tamaño $m \times m$, cuyas columnas $(u_1, u_2, \dots, u_m)$ son denominadas **vectores singulares izquierdos** y forman la base ortonormal del espacio de las **filas** de la matriz X . Estas columnas son los autovectores de la matriz XX^T .
- Σ es una matriz diagonal rectangular de tamaño $m \times n$, cuyos valores diagonales $\sigma_i = \Sigma_{ii}$ son denominados **valores singulares**, valores no-negativos y ordenados de manera descendente ($\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_k \geq 0$), donde $k = \min m, n$. Corresponden a la raíz cuadrada de los autovalores no nulos de las matrices XX^T y $X^T X$. ($\sigma_i = \sqrt{\lambda_i}$)
- V es una matriz ortogonal de tamaño $n \times n$, cuyas columnas $(v_1, v_2, \dots, v_n)$ son denominadas **vectores singulares derechos** y forman la base ortonormal del espacio de las **columnas** de la matriz X . Estas columnas son los autovectores de la matriz $X^T X$.

Si bien U y V son **teóricamente ortogonales**, diversos métodos matemáticos que buscan estimarlos prefieren aproximarlos a un cierto valor de tolerancia ϵ .

1.2. Rotación de Givens

Se consideran dos columnas de X , denotadas por x_i y x_j . Se busca una matriz de rotación G :

$$G = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \quad (2)$$

tal que las nuevas columnas x'_i y x'_j , definidas por:

$$[x'_i, x'_j] = [x_i, x_j] \begin{bmatrix} c & -s \\ s & c \end{bmatrix} \quad (3)$$

satisfagan la condición de ortogonalidad $x_i^T x_j = 0$. Aquí $c = \cos \theta$ y $s = \sin \theta$.

Para encontrar el ángulo θ que ortogonaliza las columnas, se expande el producto punto entre las nuevas columnas:

$$x_i'^T x_j' = (cx_i + sx_j)^T (-sx_i + cx_j) = 0 \quad (4)$$

Expandiendo se obtiene:

$$-csx_i^T x_i + c^2x_i^T x_j - s^2x_j^T x_i + csx_j^T x_j = 0 \quad (5)$$

Usando la conmutatividad del producto punto ($x_i^T x_j = x_j^T x_i$) y agrupando para los términos cs y $c^2 - s^2$, se pueden usar las identidades trigonométricas:

$$2cs = 2\cos \theta \sin \theta = \sin 2\theta \quad , \quad c^2 - s^2 = \cos^2 \theta - \sin^2 \theta = \cos 2\theta \quad (6)$$

Reemplazando, se obtiene,

$$(x_j^T x_j - x_i^T x_i) \frac{\sin 2\theta}{2} + x_i^T x_j \cos 2\theta = 0 \quad (7)$$

Despejando se obtiene $\tan \theta$,

$$\tan 2\theta = \frac{2x_i^T x_j}{x_i^T x_i - x_j^T x_j} \quad (8)$$

Para efectos de notación, se define $a = x_i^T x_i$, $b = x_j^T x_j$ y $c = x_i^T x_j$, tal que

$$\tan 2\theta = \frac{2c}{a - b} \quad (9)$$

Se utiliza el recíproco τ para evitar divisiones por cero cuando c es muy pequeño (*)

$$\tau = \frac{b - a}{2c} = -\frac{1}{\tan 2\theta} \quad , \quad \tan 2\theta = -\frac{1}{\tau} \quad (10)$$

Se relaciona $\tan 2\theta$ con $\tan \theta$ usando la identidad del ángulo doble.

$$\tan 2\theta = \frac{2 \tan \theta}{1 - \tan^2 \theta} = \frac{2t}{1 - t^2} \quad (11)$$

$$\Rightarrow -\frac{1}{\tau} = \frac{2t}{1 - t^2} \quad (12)$$

Con esto se obtiene la **ecuación cuadrática característica** de la rotación de Jacobi.

$$t^2 + 2\tau t - 1 = 0 \quad (13)$$

Resolviendo la ecuación se obtienen dos valores para $t = -\tau \pm \sqrt{\tau^2 + 1}$. Se busca el valor absoluto más pequeño para evitar rotaciones agresivas a las columnas. Para esto se deriva una fórmula que asegura el mínimo valor al resolver la cuadrática.

$$t_1 = -\tau + \sqrt{\tau^2 + 1} \quad , \quad t_2 = -\tau - \sqrt{\tau^2 + 1} \quad (14)$$

Para $\tau > 0$, $|t_1| < |t_2|$. En cambio, para $\tau < 0$, $|t_2| < |t_1|$. Comenzando por el caso de $\tau > 0$, se multiplica y se divide t por el conjugado:

$$t = \frac{(\sqrt{\tau^2 + 1} - \tau)(\sqrt{\tau^2 + 1} + \tau)}{\sqrt{\tau^2 + 1} + \tau} = \frac{(\cancel{\tau^2} + 1) - \cancel{\tau^2}}{\sqrt{\tau^2 + 1} + \tau} = \frac{1}{\sqrt{\tau^2 + 1} + |\tau|} \quad (15)$$

Luego, para el caso $\tau < 0$, se reescribe $-\tau - \sqrt{\tau^2 + 1}$ como $-(\sqrt{\tau^2 + 1} - |\tau|)$. Luego se sigue un proceso análogo al anterior,

$$t = -\frac{(\sqrt{\tau^2 + 1} - |\tau|)(\sqrt{\tau^2 + 1} + |\tau|)}{\sqrt{\tau^2 + 1} + |\tau|} = -\frac{(\cancel{\tau^2} + 1) - \cancel{\tau^2}}{\sqrt{\tau^2 + 1} + |\tau|} = \frac{-1}{\sqrt{\tau^2 + 1} + |\tau|} \quad (16)$$

El numerador puede escribirse como la función $\text{sign}(\tau)$ dado que es 1 para $\tau > 0$ y -1 para $\tau < 0$. El denominador es igual en ambos casos. Luego,

$$t = \frac{\text{sign}(\tau)}{|\tau| + \sqrt{\tau^2 + 1}} \quad (17)$$

siempre entregará el valor absoluto más pequeño al resolver ecuación cuadrática.

Asegurando que $t = \tan \theta$ es mínimo, se pueden obtener los valores de c y s mediante identidades trigonométricas.

$$c = \cos \theta = \frac{1}{\sqrt{1 + \tan^2 \theta}} = \frac{1}{\sqrt{1 + t^2}} \quad , \quad s = \sin \theta = \cos \theta \tan \theta = ct \quad (18)$$

Teniendo los valores de c y s , se definen las nuevas columnas x'_i y x'_j :

$$x'_i = cx_i + sx_j \quad , \quad x'_j = -sx_i + cx_j \quad (19)$$

Que cumplen con las condición de ortogonalidad.

1.3. Pseudocódigo

Algorithm 1 CUDA based One-Sided Jacobi implementation

```
1: Inputs:  
    $\mathbf{X} \leftarrow$  data matrix ( $T \times D$ );  
    $\epsilon \leftarrow$  tolerance;  
   max_sweeps  $\leftarrow$  maximum amount of sweeps;  
2: Initializations:  
    $I \leftarrow \{0, \dots, D-1\}$ , indexes of  $\mathbf{X}$ ;  
    $\mathbf{V} \leftarrow$  identity matrix of size  $D \times D$ ;  
   any_rots  $\leftarrow$  loop condition;  
3: Initialize  $\mathbf{X}, \mathbf{V}$  to Device;  
4: any_rots_host  $\leftarrow$  True;  
5: sweeps  $\leftarrow 0$ , sweep counter;  
6: while (any_rots_host and sweeps  $<$  max_sweeps) do  
7:   any_rots_host  $\leftarrow$  False;  
8:   any_rots_device  $\leftarrow$  any_rots_host, copy host value to device;  
9:   for ( $r$  from 0 to  $D-1$ ) do  
10:    Sweep  $\lll D/2$  blocks, 1 thread  $\ggg$  ( $\mathbf{X}, \mathbf{V}, I_D, \text{any\_rots\_device}$ ) execute device kernel;  
11:    synchronize threads;  
12:    circular_shift( $I$ ), shuffle indexes using circular shift;  
13:   end for  
14:   any_rots_host  $\leftarrow$  any_rots_device;  
15:   sweeps++;  
16: end while
```

Algorithm 2 Sweep kernel

```
1: Sweep( $\mathbf{X}, \mathbf{V}, I_D, \text{any\_rots\_device}$ )  
2: for each pair( $i, j$ ) in  $I_D$  do  
3:    $a \leftarrow \mathbf{X}[i] \cdot \mathbf{X}[i]$ ;  
4:    $b \leftarrow \mathbf{X}[j] \cdot \mathbf{X}[j]$ ;  
5:    $c \leftarrow \mathbf{X}[i] \cdot \mathbf{X}[j]$ ;  
6:   needs_rot  $\leftarrow (|c|/\sqrt{a \cdot b}) > \epsilon$ , rotation condition;  
7:   if needs_rot then  
8:      $\tau, t, \cos\_theta, \sin\_theta \leftarrow \text{given\_rot}(\mathbf{X}[i], \mathbf{X}[j])$ ;  
9:     update( $\mathbf{X}[i], \mathbf{X}[j]$ );  
10:    update( $\mathbf{V}[i], \mathbf{V}[j]$ );  
11:    any_rots_device = any_rots_device or 1;  
12:   end if  
13: end for
```
